EEE 475/575 Medical Image Reconstruction & Processing
Homework #1
1 March 2026, Sunday at 23:59

GUIDELINES FOR HOMEWORK SUBMISSION

- Upload your solutions to Moodle as **two separate files**:
  1) **PDF Report File:** The report should be typeset (i.e., no handwriting) and should properly present your results. In addition, the report should display the relevant sections of your code at the end of each subpart. No partial credits will be given to answers with missing codes or unjustified answers.
     At the beginning of your report, provide a section titled "GenAI Usage Disclosure" and clearly explain whether you have used any genAI tools. Include the name/version of the tool and a brief description of how it was used (see the "GenAI Policy" included below).
  2) **Code File:** You can use any programming platform of your choice. If you do not upload your code file, your homework will not be evaluated (i.e., 0 pts).
- Submission system will remain open for 1 day after the deadline:
  - No points will be lost if you submit your assignment within 12 hours past the deadline.
  - There will be a 50% penalty if you submit after 12 hours but within 24 hours past the deadline.
  - No submissions beyond 24 hours past the deadline.

GenAI Policy: Generative AI (genAI) tools (e.g., large language models, coding assistants) may be used for homework assignments and project to assist with coding and to improve writing. Any use of genAI must be clearly disclosed, including the name/version of the tool and a brief description of how it was used. Upon request, the student should be ready to provide prompts, outputs, and execution logs pertaining to their genAI use for all assignments. Students remain fully responsible for all submitted material and must be able to explain and justify their code, results, and written content. Reliance on genAI does not transfer responsibility for correctness or originality. Failure to disclose AI usage or inappropriate reliance on AI-generated content will be treated as a violation of academic integrity.

HOMEWORK #1: PREPARATION

IMPORTANT NOTE: The following codes are provided for MATLAB, but you can use any programming platform of your choice. For example, if you wish to use Python, you can first translate these codes to Python using online tools.

PREPARATION 1) Centered FFT: In medical image reconstruction, the preferred way to display the image and the k-space data (i.e., Fourier domain data) is such that the origin (i.e., $(x, y) = (0,0)$ or $(k_x, k_y) = (0,0)$) is at the center of the displayed image or k-space data. The usual convention for the FFT in MATLAB, however, is that the origin is at the beginning of the array, or the upper left corner of a 2D array (e.g., the DC point in Fourier domain data is the first

element of an array). To perform a centered 2D FFT, you need to do *ifftshift*/*fftshift* before/after the 2D FFT. You will be doing this a lot throughout the homework assignments in this class, so define the following two functions and save them as two separate files (i.e., *fft2c.m* and *ifft2c.m*):

```
function d = fft2c(im)
% d = fft2c(im)
%
% fft2c performs a centered fft2
d = fftshift(fft2(ifftshift(im)));
end

function im = ifft2c(d)
% im = fft2c(d)
%
% ifft2c performs a centered ifft2
im = fftshift(ifft2(ifftshift(d)));
end
```

When you type '*help fft2c*' in MATLAB, you will now see the commented text that gives the function usage. Pay attention to include "*help*" sections when you write your own functions. In the functions above, *ifftshift* rearranges the quadrants of the 2D array such that the element at the center is moved to the upper left corner (this is how MATLAB expects the image/data to be before an FFT/IFFT operation). On the other hand, *fftshift* rearranges the quadrants of the 2D array such that the element at the upper left corner is moved to the center (this is how we want to display the image/data). To see this, type the following in MATLAB's command line:

```
>>  A = [1 2 3 ; 4 5 6 ;  7 8 9]
>>  ifftshift(A)
>>  fftshift(A)
```

Note that *fftshift* and *ifftshift* give exactly the same result when the array size is even-valued, but are different otherwise.

You will also need centered 1D FFT, so define the following two functions and save them as two separate files (i.e., *fftc.m* and *ifftc.m*):

```
function d = fftc(im)
% d = fftc(im)
%
% fftc performs a centered fft
d = fftshift(fft(ifftshift(im)));
end

function im = ifftc(d)
% im = fftc(d)
%
% ifftc performs a centered ifft
im = fftshift(ifft(ifftshift(d)));
end
```

PREPARATION 2) Displaying images: To display a grayscale image in MATLAB, we typically use the *imshow* function. Get acquainted with this function. We typically use this function as follows:

```
>> imshow(im,[])
```

By default, `imshow(im)` without the second input argument maps pixel intensity 0 to black and pixel intensity 1 to white. The second input argument "[]" tells *imshow* to map the minimum and maximum pixel intensities in `im` to black and white, respectively. So, "`imshow(im,[])`" is a short-hand notation for either of following two lines:

```
>> imshow((im-min(im(:)))/(max(im(:))-min(im(:))))
>> imshow(im,[min(im(:)) max(im(:))])
```

In the first line, the pixel intensities in `im` are normalized and remapped to [0 1] interval, and the second input argument is omitted. In the second line, the pixel intensity range is explicitly specified as the minimum and maximum of `im`. If you use the square bracket with no values inside (i.e., `imshow(im,[])`), MATLAB understands that the minimum/maximum values of `im` should be mapped to black/white. If you type different values inside the square brackets, the first value will be mapped to black and the second value will be mapped to white.

Try the above 3 lines and the following 2 lines for yourself for one of the images in the homework and observe the differences in the displayed images:

```
>> imshow(im,[min(im(:))/2 max(im(:))/2])
>> imshow(im,[0 max(im(:))/10])
```

PREPARATION 3) Displaying k-space magnitude spectrum: To display the k-space magnitude spectrum as an image, we typically do the following:

```
>> imshow(log(abs(d)+1),[])
```

In k-space, the value at DC (i.e., at the origin of k-space) is MUCH larger than the values elsewhere. The *log* operation brings these values closer together, so that we can visualize the k-space data more easily. The addition of 1 is to avoid the *log(0)* problem.

PREPARATION 4) For quantitative image quality assessment (IQA), you can use MATLAB's built-in functions *psnr* and *ssim*. Get acquainted with these functions. When using these functions, the images to be compared need to be normalized to the same scale.

PREPARATION 5) From the Moodle page of the class, download the following file:

- ge_phantom.mat: Line-by-line acquired full k-space data for GE phantom.

Homework #1

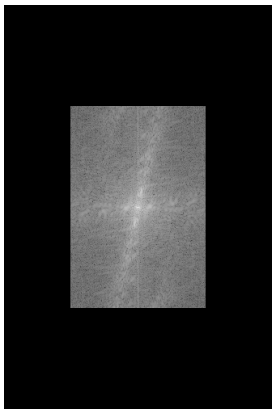
In this homework, you will get acquainted to image domain and k-space (Fourier domain), and implement partial k-space algorithm for MRI. If coded efficiently (e.g., vector arithmetic instead of *for* loops), the codes for each subpart of the homework should not take longer than 10-20 lines.

PART 1: IMAGE DOMAIN AND FOURIER DOMAIN FOR PHOTOS (20 pts)

1.1) Photo: Cameras acquire images directly in image domain. Read a photo of yourself (or any photo of your choice) into your workspace (e.g., using *imread* in MATLAB) and convert it to grayscale (e.g., using *rgb2gray* in MATLAB) and double precision (e.g., using *double* in MATLAB). Assume the resulting image has size $N_x \times N_y$. We will use this image as m_{ref}, i.e., the *reference image* for quantitative IQA. Display this image. Take centered 2D FFT of this image and display the magnitude spectrum.

1.2) Downsampled Image: Downsample the reference image by 2 in both x- and y-directions (i.e., keep every other pixel in the image in both x- and y-directions and remove the rest from the 2D array). The resulting image, m_{down2}, will have size $(N_x/2) \times (N_y/2)$, emulating a photo from a camera with lower resolution. Display this image and its magnitude spectrum.

1.3) Zero Padding in Fourier Domain: Pad the spectrum of m_{down2} with zeroes to get a spectrum of size $N_x \times N_y$. To do this, you can first create a 2D array of size $N_x \times N_y$ containing all zeroes, and then place the spectrum of m_{down2} to its center. Display the resulting zero-padded magnitude spectrum (an example is shown on the right). Take centered inverse 2D FFT to compute the corresponding image (because of Fourier domain padding, take the magnitude of the resulting image to make sure no complex values remain). Display the resulting image and comment on its quality with respect to m_{ref}. Also, display the magnitude of the error image between this image and m_{ref} (before computing the error image and IQA metrics, normalize each image by its own maximum pixel intensity). Compute PSNR and SSIM for this image with respect to m_{ref}.



1.4) Repeat Part (1.3), this time by downsampling the reference image by 4 in both x- and y-directions, to emulate a photo from a camera with much lower resolution.

1.5) Briefly comment on whether PSNR and SSIM accurately reflect your visual observations in this part of the homework.

PART 2: K-SPACE AND IMAGE DOMAIN FOR MRI (30 pts)

2.1) k-space Data: Load ge_phantom.mat. This gives the full k-space data acquired in a line-by-line fashion, with size $N_{ro} \times N_{pe}$, where N_{ro} is the number of readout (RO) samples and N_{pe} is the number of phase encode (PE) lines (both are 256 for this data). Display the magnitude spectrum. Compute the corresponding complex-valued MRI image, m_f, by taking centered inverse 2D FFT. Display the magnitude image, phase image, real part of the image, and the imaginary part of the image. We will use the magnitude image as the *reference image* for quantitative IQA (i.e., $m_{ref} = |m_f|$).

Important Note: Define a variable named max_val as the maximum pixel intensity in m_{ref} from Part (2.1). For all error images in the rest of this homework, use an *imshow* grayscale window of [0 0.2*max_val] to be able to compare error images across different results. During PSNR and SSIM computations for the rest of the homework, normalize both the image of interest and m_{ref} by max_val.

2.2) Skipping lines in k-space: Downsample the k-space data by 2 in phase-encode (PE) direction only (i.e., keep every other element in the k-space data in PE-direction and remove the rest from the 2D array). The resulting k-space data will have size $N_{ro} \times (N_{pe}/2)$, emulating the case of acquiring half the lines in k-space by skipping every other line. Display the magnitude spectrum and the corresponding magnitude image. Briefly comment on the field-of-view (FOV) and resolution of this image, and the problems that you see in the image.

2.3) The image from Part (2.2) cannot be compared with the reference image. To enable comparison, we need to make sure that both images are displayed for the same FOV. For this purpose, downsample the full k-space data by 2 in PE-direction, but this time by setting every other element to zero in PE-direction. The resulting k-space data will still have size $N_{ro} \times N_{pe}$, but every other line in k-space will contain zeroes. Display the magnitude spectrum and the corresponding magnitude image. Also, display the magnitude of the error image between this image and m_{ref}. Briefly comment on the FOV and resolution of this image, and the problems that you see in the image. Compute PSNR and SSIM for this image with respect to m_{ref}.

2.4) Repeat Part (2.3), this time by downsampling the full k-space data by 4 in PE-direction.

2.5) Acquiring fewer lines in k-space: Start with the full k-space data, and keep only the central half of the data in PE-direction (i.e., remove the first 64 lines and the last 64 lines of data). The resulting k-space data will have size $N_{ro} \times (N_{pe}/2)$, emulating the case of acquiring the central half of the lines in k-space. Display the magnitude spectrum and the corresponding magnitude image. Briefly comment on the FOV and resolution of this image.

2.6) The image from Part (2.5) cannot be compared with the reference image. To enable comparison, we need to make sure that both images are displayed with the same pixel size. For this purpose, start with the full k-space data again, keep only the central half of the data in PE-direction and set the rest of the data to zero. The resulting k-space data will still have size $N_{ro} \times N_{pe}$, but the first 64 lines and the last 64 lines in k-space will contain zeroes. Display the

magnitude spectrum and the corresponding magnitude image. Also, display the magnitude of the error image between this image and m_{ref}. Briefly comment on the FOV and resolution of this image. Compute PSNR and SSIM for this image with respect to m_{ref}.

2.7) Repeat Part (2.6), this time by keeping the central 1/4th of the data in PE-direction and setting the rest of the data to zero.

2.8) Briefly comment on whether PSNR and SSIM accurately reflect your visual observations in this part of the homework.

PART 3: PHASE-COMPENSATED MRI IMAGE (20 pts)

3.1) Phase-compensated Full k-space Image: Use the central ±1/8th of k-space data (i.e., the central 25% of k-space data corresponding to the central 128 PE lines) to estimate a low-resolution phase image, $p(x,y) = e^{j\angle m_s(x,y)}$ with size $N_{ro} \times N_{pe}$. Display the phase of this image. Next, display the phase-compensated full k-space image, i.e., $m_{pc} = \left|Re\{m_f(x,y)p^*(x,y)\}\right|$. Display the magnitude of the error image between this image and m_{ref}. Compute PSNR and SSIM for this image with respect to m_{ref}.

3.2) Repeat Part (3.1) using the central ±1/16th of k-space data for phase image.

3.3) Repeat Part (3.1) using the central ±1/32th of k-space data for phase image.

3.4) Repeat Part (3.1) using the central ±1/64th of k-space data for phase image.

3.5) Based on your results, briefly comment on the quality of the phase-compensated image as a function of central k-space data ratio used for computing the phase image (e.g., the ratio is 0.25 when we use the central ±1/8th of k-space data).

3.6) Briefly comment on whether PSNR and SSIM accurately reflected your visual observations in this part of the homework.

Note: Partial k-space reconstructions cannot provide an image with higher quality than m_{pc}, as they rely on the low-resolution phase in $p(x,y)$.

PART 4: PARTIAL K-SPACE RECONSTRUCTION (30 pts)

4.1) Zero Fill (ZF) Reconstruction: Generate 5/8th partial k-space data by retaining the first 5/8th of the PE lines, and setting the remaining k-space data to zero. Display the k-space spectrum for this partial k-space data. Display the magnitude image for the ZF (i.e., trivial) reconstruction. Display the magnitude of the error image between this reconstruction and m_{ref}. Compute PSNR and SSIM for this reconstruction with respect to m_{ref}. Comment on both the visual and quantitative results.

4.2) POCS: For 5/8th partial k-space data, implement the POCS algorithm as given in the lecture notes. As the output image for the ith iteration, use $m_{out} = |Re\{m_i(x,y)p^*(x,y)\}|$. Display the following images for the first 5 iterations: output image, k-space magnitude spectrum of the output image, the magnitude of the error image between the output image and m_{ref}. Compute PSNR and SSIM for the first 5 iterations. Comment on both the visual and quantitative results.

4.3) Comparison at lower partial k-space ratio: Generate 9/16th k-space data and display the k-space spectrum. Compare ZF and POCS for this data. Display the resulting images, k-space spectrums, and the magnitude of the error images. Compute PSNR and SSIM values. For POCS, display only the result of the 5th iteration. Comment on both the visual and quantitative results.

4.4) Repeat Part (4.4) for 17/32th k-space data.

4.5) Repeat Part (4.4) for 33/64th k-space data.

4.6) Based on your results, briefly comment on the performances of ZF and POCS as a function of the partial k-space ratio for this phantom data.

4.7) Briefly comment on whether PSNR and SSIM accurately reflected your visual observations in this part of the homework.

Final Remark: In partial k-space acquisitions used in clinical MRI scanners, the k-space ratio is typically kept above 0.6 (or even 0.7) to ensure robustness, at the expense of longer scan times.